

Project (P) - Final Report

Evaluating the Accuracy of Predicting Stargazers in Open Source Projects

Group 7

Data Engineering II

Course Code: 1TD076

Spring Term 2026 / VT 26

May 24, 2026

Bradley Deku

bradley.deku.4602@student.uu.se

Ioannis Peridis

ioannis.peridis.5471@student.uu.se

Kanza Raisi

kanza.raisi.6874@student.uu.se

Sotiris Oikonomou

sotiris.oikonomou.7076@student.uu.se

William Shaye Min

william.shaye-min.8672@student.uu.se

1 Introduction

The popularity of an online artifact is often used as a proxy for its quality, influence, or community traction. On video platforms this may be subscriber counts, views or likes. The GitHub equivalent signal is the **stargazer**, a user-facing bookmark that doubles as a public endorsement of a repository. Star counts are therefore widely interpreted as a measure of a project’s standing in the open-source ecosystem, and being able to predict them from observable repository properties is particularly useful to project maintainers, package managers, and researchers studying open-source dynamics.

This report presents an end-to-end study of star-count prediction for open-source projects. The project has three parts. First, we assemble a balanced dataset of 1,000 GitHub repositories collected via the GitHub REST API, sampled uniformly across four logarithmic star-count buckets to avoid the heavy bias toward flagship projects that a naive top-1000 query would produce. Second, we train and compare seven regression model families including a linear baseline, several gradient-boosted methods and bagged tree ensembles. The best model is selected by 5-fold cross-validation on the training split and reporting final performance on a held-out test set. R^2 is used throughout as the evaluation metric. Finally, we package the trained model into a production system. A continuous-integration pipeline trains and compares models on a development virtual machine (VM), promotes the best model to a separate production VM, and exposes it through a small FastAPI web application that ranks repositories by predicted star count.

2 Related Work

Predicting popularity from early signals is a well-established problem across social platforms. Twitter retweet cascades, YouTube view counts, and Reddit submission scores have all been analysed, with both classical regression methods and more recent deep/neural methods [1]. Previous studies have studied GitHub extensively as a source of large-scale software engineering data, with mining software repositories (MSR) venues producing datasets and methodological guidance on how to use the GitHub API responsibly and at scale [2, 3]. Repository features are often used as predictors of downstream quantities. The features most popularly used are commit frequency, contributor counts, issue activity, and language. Several studies have also specifically used star counts as predictors of project popularity and/or success [4, 5].

Recent empirical comparisons consistently find that gradient boosting methods trees and random forests outperform deep neural networks on small-to-medium tabular datasets that have mixed numeric and categorical features [6, 7]. This finding motivates our choice to focus on tree-based methods (Random Forest, Extra Trees) and three gradient-boosting variants alongside linear baselines, and to drop the MLP regressor from the final comparison.

3 System Architecture

3.1 Overview and Architecture Diagram

The system has two largely independent subsystems (see Figure 1). An offline data collection subsystem runs on four virtual machines on the Swedish Science Cloud (SNIC UPPMAX) and produces the training dataset by querying the GitHub REST API in parallel. A training and deployment subsystem consumes the resulting CSV, trains and compares seven regression models on a development VM, and promotes the best model to a separate production VM, where the Star Predictor application serves repository rankings.

3.2 Data Collection

To build the dataset, we queried the GitHub REST API using a stratified uniform sampling strategy. Because star counts follow a heavy power-law distribution, a naive top-1000 query would over-represent repositories with similar star counts. Instead, we divided the star spectrum into four logarithmic buckets

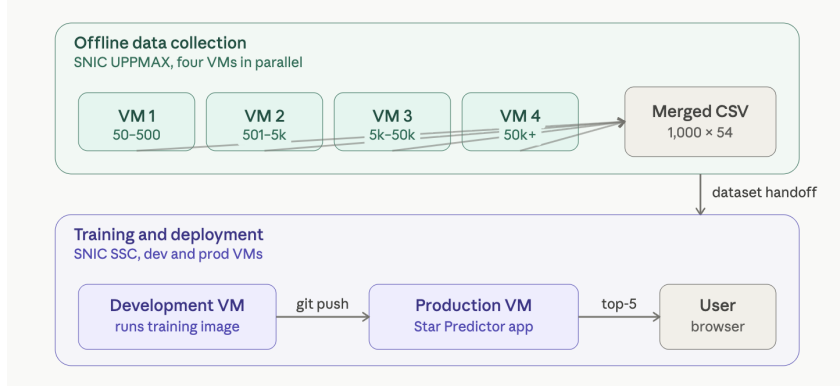


Figure 1: Architecture Diagram

— 50–500, 501–5,000, 5,001–50,000, and 50,001+ — and collected exactly 250 repositories per bucket, yielding a balanced 1,000-row dataset spanning five orders of magnitude.

Collection ran in parallel, one VM per bucket, reducing total wall-clock time from roughly 8–12 hours to approximately 2–3 hours (a 4x speedup). Each VM ran a Python script (`collect_uniform.py`) that operates in two phases: a **Search phase** that identifies candidate repositories via the GitHub Search API, and an **Enrich phase** that makes approximately 13 additional API calls per repository to collect deep metadata — commits, contributors, releases, issues, pull requests, CI/CD workflows, language breakdown, community health files, and owner profile.

After collection, the four CSV files were merged with a separate script (`merge_csv.py`). Duplicate rows caused by restarts on overlapping ranges were replaced with repositories from a pre-collected overflow pool, preserving the exact 250-per-bucket balance. The final dataset contains 1,000 rows, 1,000 unique repository IDs, and 54 features. The subsequent subsections describe the feature pipeline and model training, the deployment infrastructure, and the Star Predictor application.

3.3 Feature Engineering and Model Training

3.3.1 Feature Engineering

The features cover basic metadata (language, forks, issues, size, license, etc.), activity metrics (commit count, contributor count, release count, pull requests), language breakdown, CI/CD presence, community health files, commit activity (weekly averages), owner profile, and engineered date-derived features (repository age, days since last push/update, topic count). The target column is stargazers count.

From the 54 raw columns, we drop the target and its leakage variants (`watchers_count`, `star_category`), identifiers, and other constant or redundant columns. Raw timestamps are also replaced by engineered features. Categorical features are one-hot encoded and missing values are handled by using median imputation or most-frequent imputation. This leaves 44 input features (40 numeric/boolean and 4 categorical).

Because star counts span five orders of magnitude (3.2), models are trained on $\log(1 + \text{stargazers_count})$ and predictions are inverted before reporting.

3.3.2 Model Training and Selection

The 1,000 repositories are split into train (850) and test (150) sets. Hyperparameters are tuned by 5-fold cross-validation on the training set using GridSearchCV, with selection by mean CV R^2 . The held-out test set is used only for final generalisation reporting. Seven models are compared: Linear, Ridge, Random Forest, Extra Trees, Gradient Boosting, Histogram (Hist) Gradient Boosting, XGBoost and Multi-Layer Perceptron (MLP).

Training runs inside a Docker image and emits the serialised best-model pipeline, feature manifest, full comparison table, and exact train/test splits. These are the inputs to deployment.

3.4 Deployment

The deployment subsystem runs on two VMS. There is a development node that retrains models and a production node that runs the user-facing application. Both are provisioned by a Python script that create the servers, attaches floating IPs, and emit an Ansible inventory describing them. Two Docker images are published to Docker Hub: a development image containing the training pipeline, and a production image containing the Star Predictor application.

Model promotion is handled through a Git-based pipeline using a server-side post-receive hook. The development node holds a working Git repository for the model artefacts. The production node holds a corresponding bare repository, configured with a post-receive hook that fires whenever new commits arrive, and trusts the development node’s SSH key. When a new model is trained, the development node commits the resulting artefacts and pushes them to the production bare repository. The hook then checks them out into a directory that is already mounted into the running production container, and restarts that container so it picks up the new model on the next request. The push is idempotent, in that no commit is created when the artefacts are byte-identical to the deployed ones. The full set of Ansible playbooks is checked in to the project repository, so the entire pipeline can be re-created on fresh VMs by running a fixed sequence of ansible-playbook invocations.

3.5 Application

The production application is a small FastAPI web service that demonstrates the trained model end-to-end. It exposes a single-page browser frontend with a "Show random top 5 ranking" button (see in appendix B), alongside a health endpoint that reports whether the model and supporting files are available, and a prediction endpoint that produces the ranking shown to the user.

The prediction path reuses the same feature engineering code as training, guaranteeing that pre-processing at inference time is identical to that at training time. Each click of the demo button samples five repositories from the held-out test set, runs them through the feature pipeline, invokes the trained model and returns the five repositories ordered by predicted star count (see figure 2).

Predicted Ranking			5 sampled repositories
RANK	REPOSITORY	PREDICTED STARS	ACTUAL STARS
#1	Chalarangelo/30-seconds-of-code	112,716	127,885
#2	ansible/ansible	69,149	68,618
#3	ccfos/nightingale	17,771	13,029
#4	coder/coder	12,311	13,260
#5	TencentBlueKing/bk-lite	321	145

Figure 2: Showing random top 5 ranking

4 Results

4.1 Scalability Analysis

The collection architecture is horizontally scalable by design. Each VM is fully independent. It receives its star-count range as command-line arguments, maintains its own checkpoint file, and rotates its own pool of API tokens. No inter-VM communication is required. Scaling to more VMs is straightforward. A finer-grained bucket split (e.g. eight buckets instead of four) allows eight VMs to collect in parallel with no code changes to collect_uniform.py.

The bottleneck is the GitHub API rate limit per token, rather than compute resources. Each VM spends the majority of its time sleeping between API calls. A single-VM sequential collection of the same 1,000 repositories would require approximately 8–12 hours. The parallel deployment of four VMs reduces this to approximately 2–3 hours, achieving a 4x speedup proportional to the number of VMs.

Hyperparameter tuning was ran on a single development VM via GridSearchCV, which was sufficient at this dataset size. Scaling to a larger search space or dataset would warrant distributing the search across multiple VMs, e.g. with Ray Tune.

4.2 Model Comparison

Random Forest, Hist Gradient Boosting, XGBoost, and Gradient Boosting all fall within 0.02 of one another on CV mean R^2 , well inside one standard deviation across folds. Random Forest was selected on its marginal test-set advantage (0.633 vs. 0.623 for Hist Gradient Boosting), but the gap is unlikely to be statistically meaningful. The best hyperparameters for the selected model are `{max_depth: 12, max_features: 0.7, min_samples_leaf: 2, min_samples_split: 5}`.

Model	Train R^2	CV R^2	Test R^2	Time (s)
Random Forest*	0.926	0.739 ± 0.040	0.633	184.400
Hist Gradient Boosting	0.862	0.722 ± 0.066	0.623	70.300
XGBoost	0.904	0.720 ± 0.106	0.600	59.000
Gradient Boosting	0.856	0.722 ± 0.063	0.567	166.900
Extra Trees	0.940	0.558 ± 0.102	0.559	97.400

Table 1: Cross-validation and test performance of the five tree-based models. The selected best model is marked with *. Cross-validation R^2 is reported as mean $\pm$ standard deviation across folds.

Figure 3 reframes this as an accuracy-cost trade-off. Hist Gradient Boosting trains in 70s against Random Forest’s 184s, making it the more attractive choice whenever retraining cost matters. Extra Trees and Gradient Boosting are dominated on both axes and can be dropped from further consideration. All tree models show a train-test gap of $0.24 - 0.38 R^2$, indicating substantial overfitting. The two linear baselines (Ridge and Linear Regression) failed catastrophically, returning large negative R^2 on both cross-validation and the test set. Same goes for the MLP regressor, as the dataset was too small for it to learn and the training was too unstable. See Appendix A for the graph.

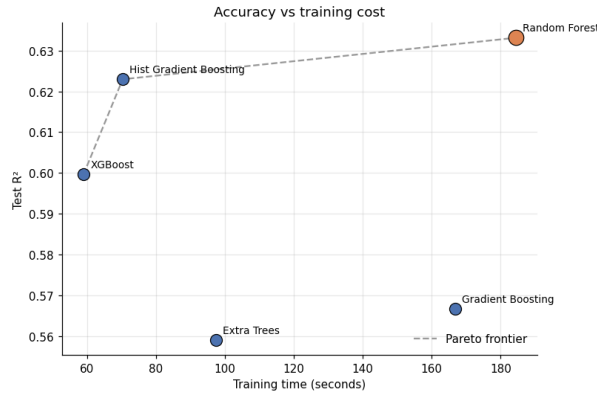


Figure 3: Test R^2 versus training time for the tree-based models.

5 Conclusion

Tree ensembles significantly outperformed the baseline linear models, confirming that the relationship between repository features and popularity is strongly non-linear, and the small train-test gap relative to other ensembles suggests Random Forest generalises reasonably well at this dataset size.

Limitations include the modest dataset (1,000 repositories) and the absence of an accuracy-based promotion gate in the deployment pipeline. Future extensions could be to enlarge and balance the dataset to distribute hyperparameter tuning across multiple VMs (e.g. with Ray Tune) once the search space grows beyond what fits on a single dev node. One may also add a model-quality check that blocks promotion when a new model regresses against the currently deployed one.

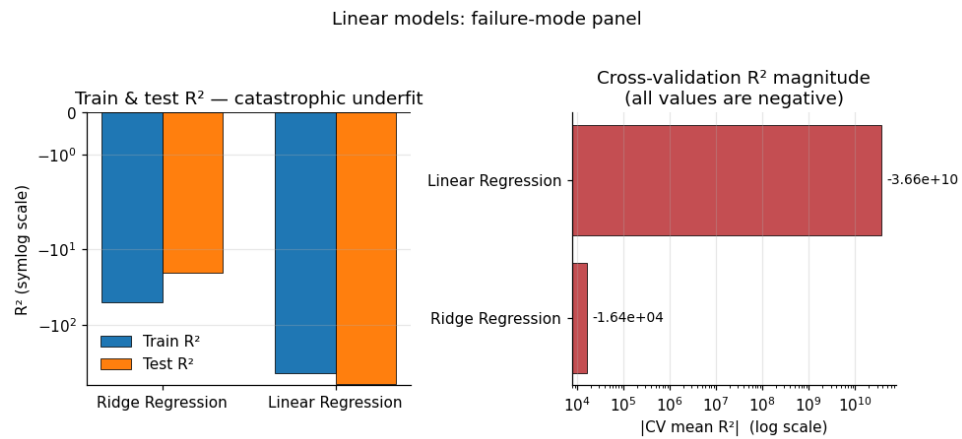
References

- [1] G. Szabó and B. Huberman, “Predicting the Popularity of Online Content,” *Communications of the ACM*, vol. 53, Dec. 2008, doi: 10.2139/ssrn.1295610.
- [2] O. Dabic, E. Aghajani, and G. Bavota, “Sampling Projects in GitHub for MSR Studies.” 2021. [Online]. Available: <https://arxiv.org/abs/2103.04682>
- [3] E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D. M. German, and D. Damian, “The promises and perils of mining GitHub,” in *Proceedings of the 11th Working Conference on Mining Software Repositories*, in MSR 2014. New York, NY, USA: Association for Computing Machinery, 2014, pp. 92–101. doi: 10.1145/2597073.2597074.
- [4] H. Borges, A. Hora, and M. T. Valente, “Understanding the Factors That Impact the Popularity of GitHub Repositories,” in *2016 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, IEEE, Oct. 2016, pp. 334–344. doi: 10.1109/icsme.2016.31.
- [5] H. Borges and M. Valente, “What’s in a GitHub Star? Understanding Repository Starring Practices in a Social Coding Platform,” *Journal of Systems and Software*, vol. 146, pp. 112–129, Sep. 2018, doi: 10.1016/j.jss.2018.09.016.
- [6] L. Grinsztajn, E. Oyallon, and G. Varoquaux, “Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?,” Jan. 2022, pp. 507–520. doi: 10.52202/068431-0037.
- [7] R. Shwartz-Ziv and A. Armon, “Tabular data: Deep learning is not all you need,” *Inf. Fusion*, vol. 81, no. C, pp. 84–90, May 2022, doi: 10.1016/j.inffus.2021.11.011.

6 Appendix

A Linear-model failure mode

Both linear baselines produced predictions far worse than the constant mean.



B Star Prediction App

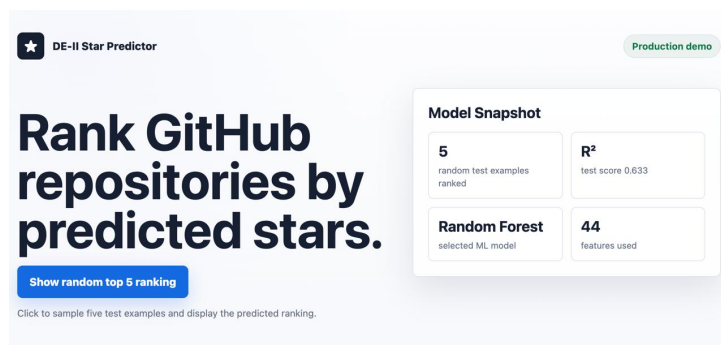


Figure 4: Home Page of Star Predictor App

C Useful Links

GitHub Repository: <https://github.com/Secon01/de2-project3>